

TRM-MCP: Efficient Context with DRAM-trained Retrieval

Patrick Dugan
Morality Lab

May 6, 2026

Abstract

Model Context Protocol (MCP) surfaces expose tools, resources, templates, and structured memory, but a small model can waste most of its active context repeatedly scanning the same lookup surface. We propose TRM-MCP: a decomposition in which Tiny Recursive Models (TRMs) learn where to look, which handle or template to use, whether retrieved evidence is relevant, and when to stop. The current artifact set includes a generic MCP trace matrix across filesystem, GitHub, and Postgres-like surfaces with 15 traces and 42 route/retrieve/verify rows; a Primehub schema-retrieval live study on Qwen3.5-9B; and a storyworld play-diary MCP environment study over 15 scenarios and 20 plays. The Primehub study shows retrieval-assisted schema memory improving exact-structure reward on `ascii_tree` from 0.0 to 0.8 and `pydantic_adherence` from 0.0 to 1.0, while using more tokens in this implementation. The storyworld study shows average diary lift of +0.619333 and recovery from four negative-NAV cases. These results motivate a concrete research program: train small controllers to emit retrieval pointers and stop/escalate decisions over structured external memory, optimizing capability per token rather than raw generation alone.

1 Thesis

Model Context Protocols (MCPs) had limited early success due to the token-cost of a model reasoning about where and how to retrieve data, the chain-of-thought tokens could be cleared once the curated context was assembled from database queries, but at economic cost. Our central claim is that MCP should be treated as an addressable memory surface for training Tiny Recursive Models (TRMs) to do the retrieval for the LLM. A TRM does not need to know the contents of the filesystem, repository, database, log history, or world model index. It needs to learn a compact policy:

1. Which MCP family should be queried?
2. Which URI, template, entity, or handle is likely relevant?
3. Is the selected resource an exact hit, a near miss, wrong scope, or wrong family?
4. Has enough evidence been retrieved?
5. Should the system stop, continue, or escalate?

This makes TRM-MCP a control-plane architecture rather than a knowledge-distillation architecture. The TRM outputs pointers and actions, not prose answers. The language model remains useful for synthesis, but exact lookup and verification are shifted into typed, trainable control surfaces.

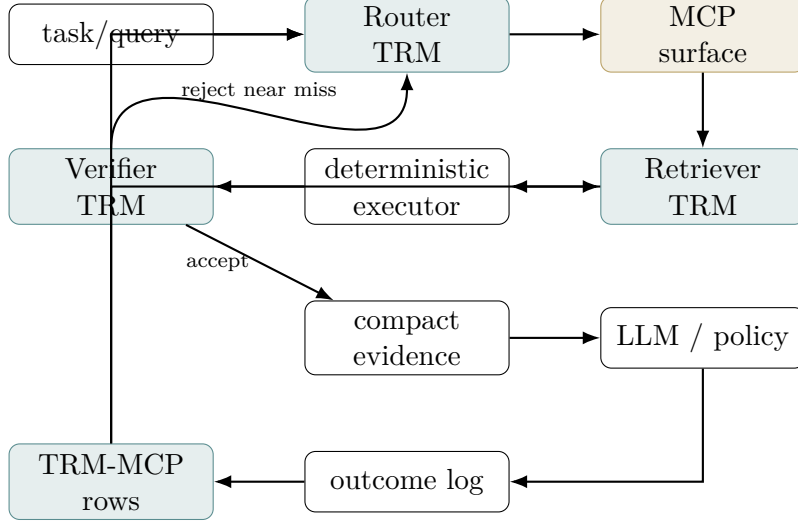


Figure 1: TRM-MCP treats the MCP layer as structured external memory. Small controllers emit lookup pointers and verifier decisions; the deterministic executor performs the actual resource access.

This decomposition follows the same compactification logic as HRM and TRM-style recursive reasoning models [5, 10], but applies it to context access rather than puzzle solving alone. MCP is an open standard for connecting AI applications to external systems and data sources [7]. Recent MCP literature frames the protocol as an interoperability layer with a new security and maintainability surface [2–4]. MeTTa and OpenCog Hyperon motivate the symbolic side of the design: handles, answer shapes, failure modes, and validators can be represented as editable metagraph objects rather than hidden prompt prose [1, 6]. In the Hermes setting, this connects to tool-use and skill-harness work on instruction-tuned agents [9] and verifier-oriented environments such as Prime Intellect’s Verifiers MCPEnv interface [8].

2 Architecture

TRM-MCP decomposes lookup into four roles: an Index TRM that compresses resource labels and query cues into family and answer-shape tags; a Router TRM that selects an MCP family and lookup mode; a Retriever TRM that chooses a URI, template, or parameter sketch; and a Verifier TRM that rejects near misses, wrong-scope resources, and wrong-family matches. The deterministic MCP executor remains outside the model. It lists resources, reads resources, expands templates, and enforces token budgets.

3 Generic MCP Trace Matrix

The generic example matrix covers three MCP-like surfaces: filesystem, GitHub, and Postgres. Each pack has five traces and fourteen TRM rows. The merged matrix has 15 traces, 42 rows, 15 route rows, 12 retrieve rows, 15 verify rows, 36 exact-positive rows, and 6 negative rows. These traces exercise exact lookup and hard negatives: semantically attractive resources that should be rejected because the scope, family, or answer shape is wrong.

This is a training-corpus seed, not a trained neural TRM result. Its value is schema design: it defines row families, labels, negatives, and action surfaces for evaluating retrieval controllers.

Pack	Traces	Rows	Route	Retrieve	Verify	Negatives
filesystem	5	14	5	4	5	2
GitHub	5	14	5	4	5	2
Postgres	5	14	5	4	5	2
merged	15	42	15	12	15	6

Table 1: Generic MCP row matrix used as a training-corpus seed for route/retrieve/verify controllers.

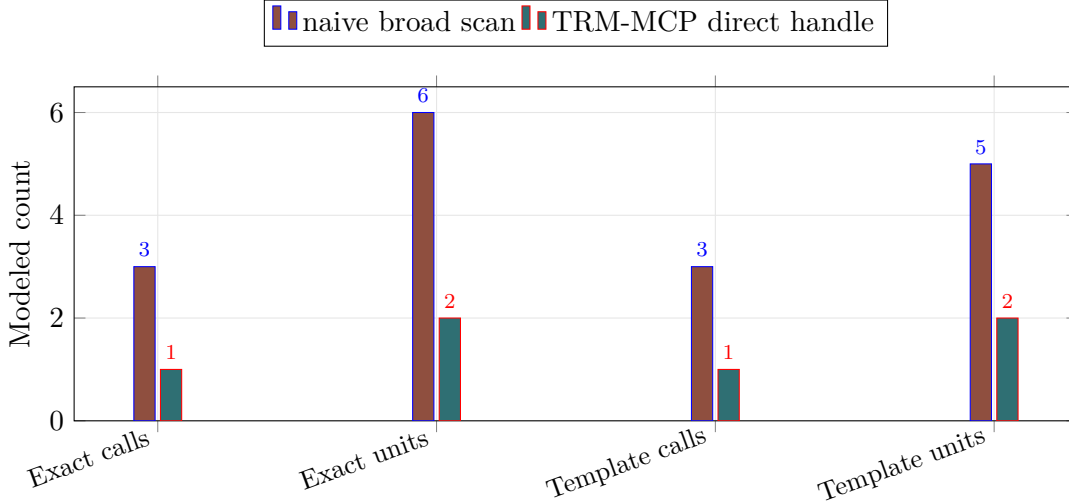


Figure 2: Analytical DB lookup model from Postgres trace design. This is not a measured latency or token benchmark; it expresses the intended control-plane advantage of direct handles over broad schema scans.

4 DB Lookup Efficiency Model

The Postgres worked example motivates an analytical efficiency model for typed lookup. For exact schema reads, the naive workflow is modeled as: list schema groups, list candidate relations, then read the selected relation. The TRM-MCP workflow is modeled as: route directly to the expected resource handle and read it. This reduces modeled MCP calls from 3 to 1 and lookup units from 6 to 2. For query-template reads, modeled MCP calls fall from 3 to 1 and lookup units from 5 to 2.

5 MeTTa Schema-Surface Enrichment

A Prime Intellect Environments hub schema MCP pack extends the same idea from SQL-style relation lookup to benchmark schema lookup. Prime Intellect’s Verifiers documentation includes an `MCPEnv` interface for connecting MCP servers to evaluation environments [8]. A script in the MeTTa programming language defines the MCP schema as a world model to constrain what data trains the retrieval TRMs. The generated surface covers `psycho_bench`, `ascii_tree`, and `pydantic_adherence`. The enriched schema surface contains 6 stable resource handles, 6 answer-shape tags, 6 query-cue sets, 6 minimal examples, 6 failure-mode sets, 2 validation-path or validator-note enriched resources, and 6 source replay links.

The schema surface is produced as a small symbolic contract, not as a long prompt. A representative excerpt of the MeTTa-style atoms used to frame the MCP database surface is:

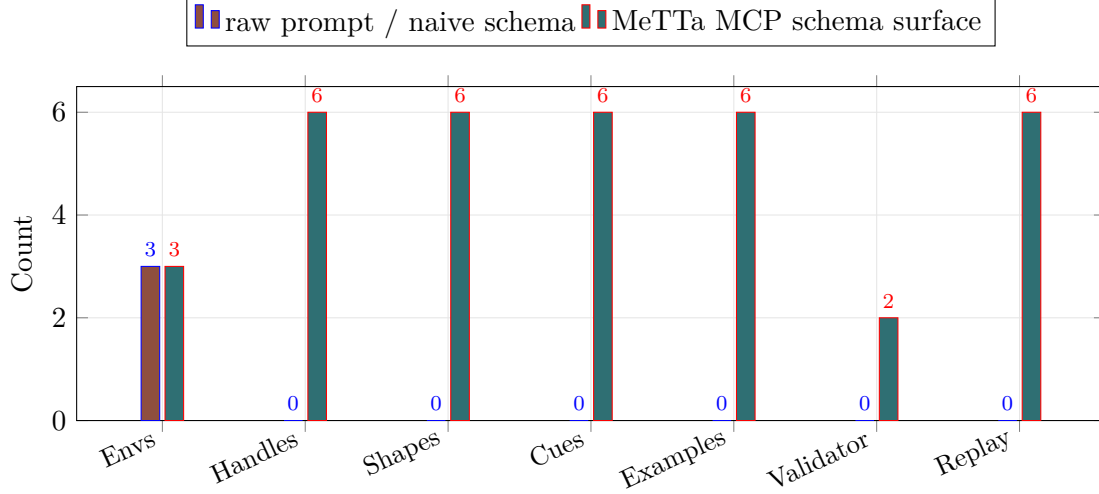


Figure 3: Schema-surface enrichment from the Primehub schema MCP pack. The current claim is not a live SQL migration; it is that MeTTa/MCP turns vague prompt and replay material into addressable schema records for routing and verification.

```
(mcp-surface "primehub_schema")
(resource-handle "primehub_schema" "ascii_tree"
  "mcp://primehub_schema/env/ascii_tree")
(answer-shape "ascii_tree" "ascii_formatted_tree")
(query-cue "ascii_tree" "ASCII tree")
(query-cue "ascii_tree" "<ascii_formatted>")
(failure-mode "ascii_tree" "missing wrapper tags")
(failure-mode "ascii_tree" "extra prose before or after the tree")

(resource-handle "primehub_schema" "pydantic_adherence"
  "mcp://primehub_schema/env/pydantic_adherence")
(answer-shape "pydantic_adherence" "strict_json_object")
(validation-path "pydantic_adherence"
  "extract_last_json -> model_validate_json(strict=False)")
(verifier-note "pydantic_adherence"
  "cooldown accepts integer day counts or numeric strings")
```

This is the concrete sense in which MeTTa improves the MCP schema: it turns benchmark prompt fragments, examples, verifier notes, and replay evidence into addressable records that can be routed to, retrieved, and rejected by small controllers.

6 Recovered Live Schema-Retrieval Measurement

The April Primehub structured-map retrieval study contains the primary empirical result for the schema-memory claim. The run compares three arms: **baseline**, with no Hermes structured-map prompt; **plain_structured_map**, with the structured-map skill prompt but no retrieved schema memory; and **retrieval_assisted**, with the structured-map prompt plus TRM-MCP-style Primehub schema memory.

Environment	Arm	Reward	Total tokens	Reward / 1k tokens
ascii_tree	baseline	0.0000	497	0.0000
ascii_tree	plain	0.0000	656	0.0000
ascii_tree	retrieval	0.8000	939	0.8520
psycho_bench	baseline	3.3283	1119	2.9744
psycho_bench	plain	3.3061	1239	2.6684
psycho_bench	retrieval	3.3311	1534	2.1715
pydantic_adherence	baseline	0.0000	1243	0.0000
pydantic_adherence	plain	0.0000	1364	0.0000
pydantic_adherence	retrieval	1.0000	1860	0.5376

Table 2: Recovered live Qwen3.5-9B schema-retrieval measurements. Retrieval-assisted schema memory improves exact-structure reward while using more tokens in this implementation.

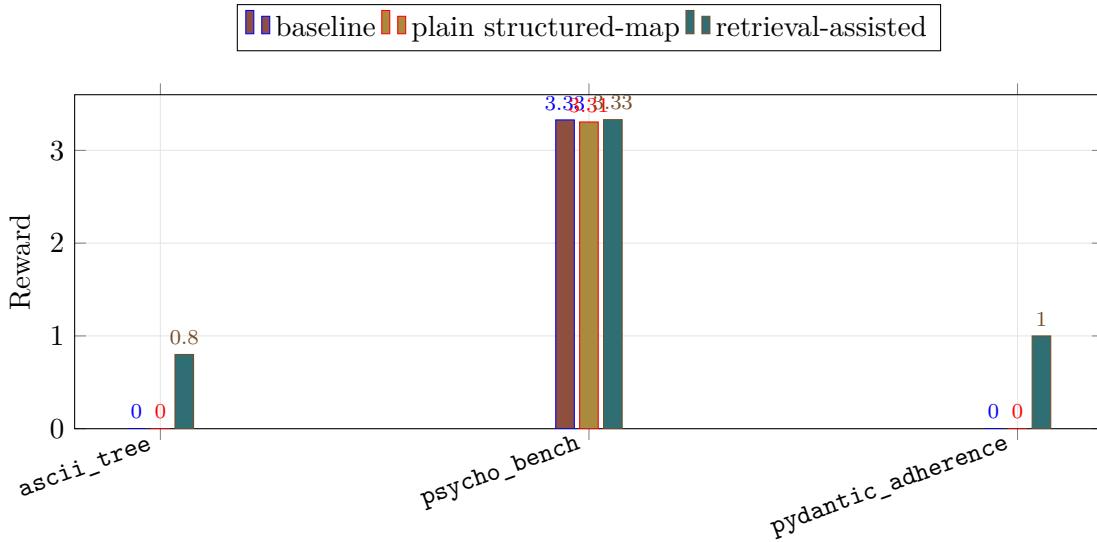


Figure 4: Measured reward by arm. Retrieval-assisted schema memory unlocks exact structured validity on `ascii_tree` and `pydantic_adherence`; `psycho_bench` was already near-saturated.

7 Storyworld Play-Diary MCP

The storyworld player adds episodic memory. Repeated playthroughs write a diary containing episode summaries, turn rows, legal actions, selected actions, NAV recommendations, endings, scores, and TRM labels. The diary lookup is subordinate to current legal actions: a retrieved action must not be copied unless it appears in the current legal-action set.

The `overnight_policy_v2` run used the `paper` profile with 15 scenarios, 20 plays, max depth 7, and branch limit 1024. The v2 policy seeds MCP memory with deterministic and NAV rollouts, then lets retrieved outcome evidence override NAV when a remembered legal action has a better score. Average diary lift reached +0.619333. The largest positive lifts were `market_square` (+4.4), `medicine_dilemma` (+1.4), `multi_agent_release_council` (+1.64), and `scarlett_letter` (+1.85). Four cases where NAV was negative were recovered back to baseline by diary memory.

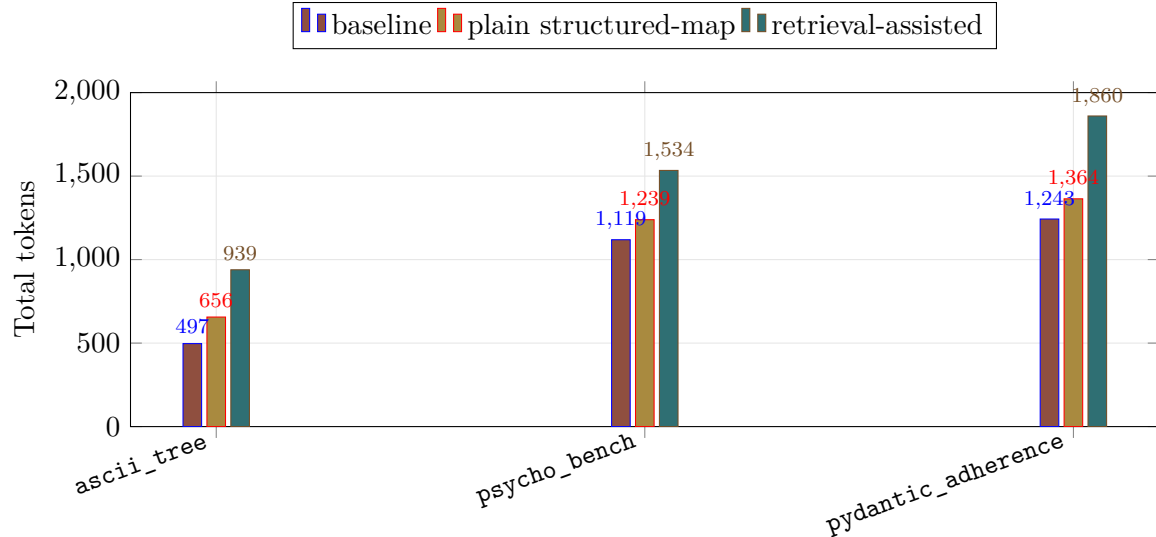


Figure 5: Token use by arm. The recovered run does not support a token-reduction claim; retrieval-assisted schema memory spends more tokens but crosses validity thresholds where baseline and plain prompting fail.

8 Training Target

TRM-MCP training rows should be state-to-retrieval-policy examples, not question-to-answer examples. A minimal row contains task text, compact task state, available MCP families, candidate descriptors, gold route or URI/template, verifier label, retrieved token count, and downstream success or failure. The target should be a structured action such as:

```
{
  "family": "storyworld_play_diary",
  "lookup": "turn_history",
  "keys": ["scarlett_letter", "Hester", "MarketScaffold"],
  "top_k": 3,
  "compression": "summary_atoms",
  "decision": "retrieve_then_answer"
}
```

The most important design choice is pointer output. Natural-language router output wastes tokens and is harder to verify.

9 Metrics

Retrieval and downstream metrics should be reported separately: first useful hit rate, MCP calls per task, tokens loaded per task, verifier rejection precision on near misses, solved-task latency, answer or decision quality, quality per 1k retrieved tokens, and catastrophic miss rate. The main scaling metric should be quality-adjusted success per token, not raw success alone.

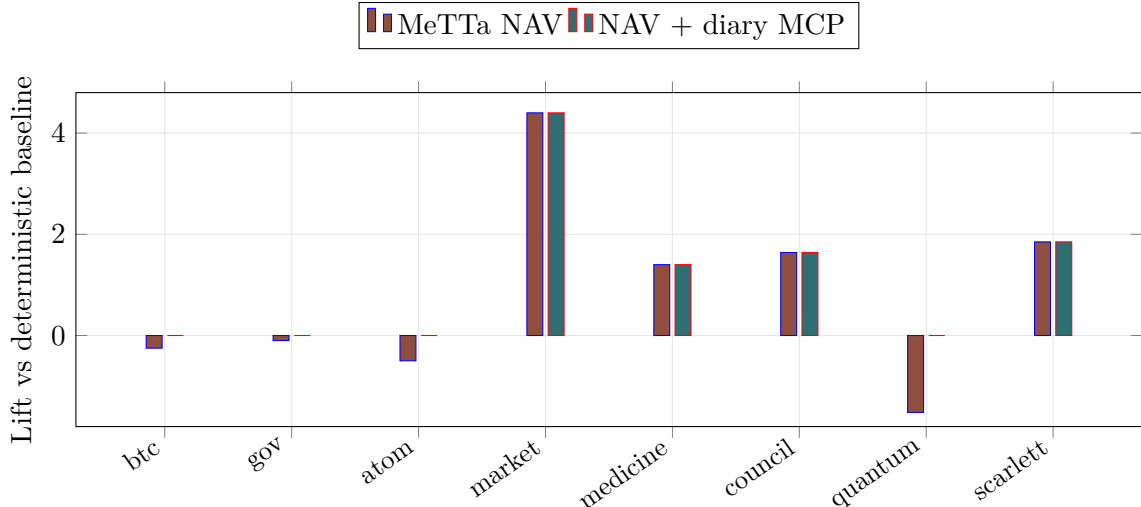


Figure 6: Selected storyworld play-diary MCP lift. Diary memory preserves positive NAV gains and recovers four negative-NAV cases back to baseline.

10 Limitations

The generic MCP matrix is currently a curated row set, not a trained neural TRM evaluation. The storyworld diary result is an environment-study result under deterministic/NAV/diary policies. The recovered Primehub schema measurement shows exact-structure validity lift but does not show token savings; retrieval-assisted schema memory used more tokens in that implementation. The Postgres lookup graph is analytical, not a measured latency result. These boundaries are important because the paper’s strongest current claim is about structured validity and control-plane decomposition, not universal context-cost reduction.

11 Conclusion

TRM-MCP reframes context as a control problem. The immediate evidence does not show that retrieval is automatically cheaper; the recovered Primehub run spends more tokens. It shows something more specific and more useful: when memory is converted into typed handles, answer shapes, examples, failure modes, and verifier notes, small controllers can be trained to retrieve compact evidence rather than asking the language model to repeatedly rediscover the same schema.

This is forward-looking for small models. A compact LLM can remain the prose synthesizer or creative policy, while retrieval, stop/escalate, legality checks, and near-miss rejection move into small trainable modules over MCP memory. The storyworld player MCP used in the Hermes hackathon work is the most concrete version of that idea: repeated play writes diary rows, retrieval brings back relevant outcomes, and current legal actions still gate what the model is allowed to reuse. If this pattern generalizes, small models do not need to become frontier-scale context reasoners to improve; they need reliable typed memory access, cheap verifiers, and controllers trained on the traces of their own failures.

A Appendix: Router and Verifier TRM Study

The first follow-on study trains a small router/verifier TRM on the generic matrix plus expanded hard negatives, then evaluates on held-out MCP families. The key metric is not answer prose quality; it is exact family selection, first useful hit, and verifier rejection precision on attractive near misses.

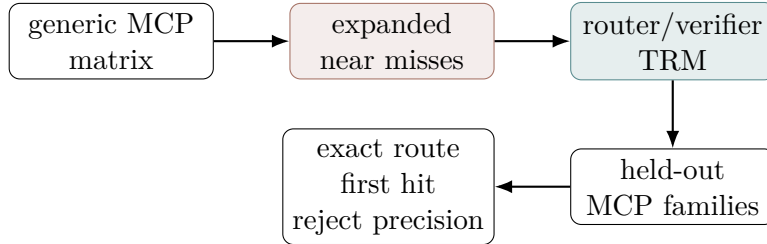


Figure 7: Router/verifier TRM study over held-out MCP families. Expanded negatives test whether the controller rejects plausible but wrong handles before they enter context.

B Appendix: Storyworld Retrieval-TRM

The second study builds a retrieval-TRM from storyworld play-diary rows and tests repeated play under fixed token budgets. This extends the Hermes hackathon storyworld player MCP: the player accumulates diary rows over playthroughs, retrieves prior outcomes for the current encounter, and only reuses an action if it is legal in the current state.

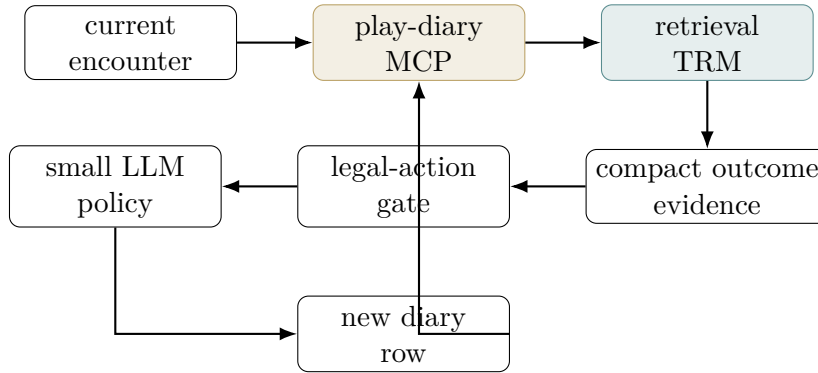


Figure 8: Storyworld retrieval-TRM study. Diary MCP memory improves repeated play only if retrieved outcomes are filtered through current legal actions and score evidence.

C Appendix: Context-Efficiency Matrix

The third study compares five context strategies under fixed token budgets: stuffed context, embedding retrieval, rules-only retrieval, TRM-MCP retrieval, and strong-LLM retrieval planners. The outcome should be reported as downstream quality per 1k input tokens, plus catastrophic miss rate and verifier rejection precision.



Figure 9: Context-efficiency matrix design. Values are placeholders for the follow-on experiment, included to define the reporting shape: quality and quality-per-token should be separated.

Artifact Availability

The bundled CSV files in `tables/` contain the local values used for the manuscript tables and plots. The principal source artifacts are listed in `evidence_manifest.json` in the paper seed directory. The current Overleaf package includes source tables and generated figure source, but not the full local replay logs.

References

- [1] Ben Goertzel. Reflective metagraph rewriting as a foundation for an agi language of thought, 2021. URL <https://arxiv.org/abs/2112.08272>.
- [2] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, 2025. URL <https://arxiv.org/abs/2506.13538>.
- [3] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions, 2025. URL <https://arxiv.org/abs/2503.23278>.
- [4] Charoes Huang, Xin Huang, Ngoc Phu Tran, and Amin Milani Fard. Model context protocol threat modeling and analyzing vulnerabilities to prompt injection with tool poisoning, 2026. URL <https://arxiv.org/abs/2603.22489>.
- [5] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL <https://arxiv.org/abs/2510.04871>.
- [6] Lucius Gregory Meredith, Ben Goertzel, Jonathan Warrell, and Adam Vandervorst. Meta-metta: An operational semantics for metta, 2023. URL <https://arxiv.org/abs/2305.17218>.

- [7] Model Context Protocol. What is the model context protocol? <https://modelcontextprotocol.io/docs/getting-started/intro>, 2026. Official MCP documentation. Accessed 2026-05-06.
- [8] Prime Intellect. Verifiers documentation: Environments and mcpenv. <https://docs.primeintellect.ai/verifiers/environments>, 2026. Documents MCPEnv support for connecting MCP servers to verifier environments. Accessed 2026-05-06.
- [9] Ryan Teknium, Jeffrey Quesnelle, and Chen Guang. Hermes 3 technical report, 2024. URL <https://arxiv.org/abs/2408.11857>.
- [10] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model, 2025. URL <https://arxiv.org/abs/2506.21734>.